

Day 30: Partial Dependence and ICE Curves

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 5, Day 3

Day 29 visualized SHAP values — real, observed attributions from real rows. Today asks a different question: what would the model predict if one feature were different, holding everything else fixed? That's a simulation, and it needs its own tools — Partial Dependence Plots (PDP) and Individual Conditional Expectation (ICE) curves. Every topic below includes the actual chart or table this lesson's fitted model produced, not just a description of it.

1. PDP/ICE vs SHAP Dependence Plots — Simulation vs Observation

Topic specification: SHAP dependence plots (Day 29) show each row's actual computed contribution — real attributions derived from the model's internal tree structure on real data. PDP and ICE, by contrast, are simulations: they artificially overwrite a feature's value across the dataset and observe how the model's predictions respond.

Explanation: Because PDP/ICE manipulate the data rather than reading attributions off it, they can ask 'what if this passenger were twice as old' even for combinations of feature values that never actually appeared in the training data — something a SHAP dependence plot, which only ever shows real observed rows, cannot do.

Real-world scenario: A pricing team wants to know what would happen to conversion probability if they raised every customer's shown price by \$10, even for customers who never actually saw that price point historically. PDP answers this directly by simulating it; SHAP can only describe what happened at prices customers actually saw.

Implementation:

```
# SHAP dependence plot (Day 29): reads real per-row attributions
shap.plots.scatter(explanation[:, "num__Fare"])

# PDP (today): simulates by overwriting Fare across every row
PartialDependenceDisplay.from_estimator(
    pipeline, X_train, features=["Fare"], kind="average")
```

Line-by-line explanation

- The SHAP line reads values that were already computed once, during Day 29's explainer call — no re-prediction happens when the plot is drawn.
- The PDP line triggers fresh predictions internally: for every grid value of Fare, every row in X_train gets that Fare value substituted in, and the pipeline re-predicts on all of them.
- This is why PDP/ICE are more computationally expensive than reading a SHAP plot that's already computed — the simulation runs a full batch of predictions per grid point.

Why choose this? PDP/ICE can explore hypothetical feature combinations the training data never actually contained, and their 'what if' framing is often more intuitive to explain to a non-technical

audience than an attribution concept like Shapley values.

Why NOT this (tradeoffs)? The simulation can produce unrealistic combinations (e.g., a very old passenger with a very high SibSp count that never co-occurs in real data), and the averaging in PDP can hide important row-level variation that ICE (Topic 3) or SHAP's per-row view already exposes.

2. Partial Dependence Plot (PDP)

Topic specification: `PartialDependenceDisplay.from_estimator(pipeline, X_train, features=[feature], kind="average")` sweeps one feature across a grid of values, holding every other feature at its real observed value, and plots the average predicted probability at each grid point.

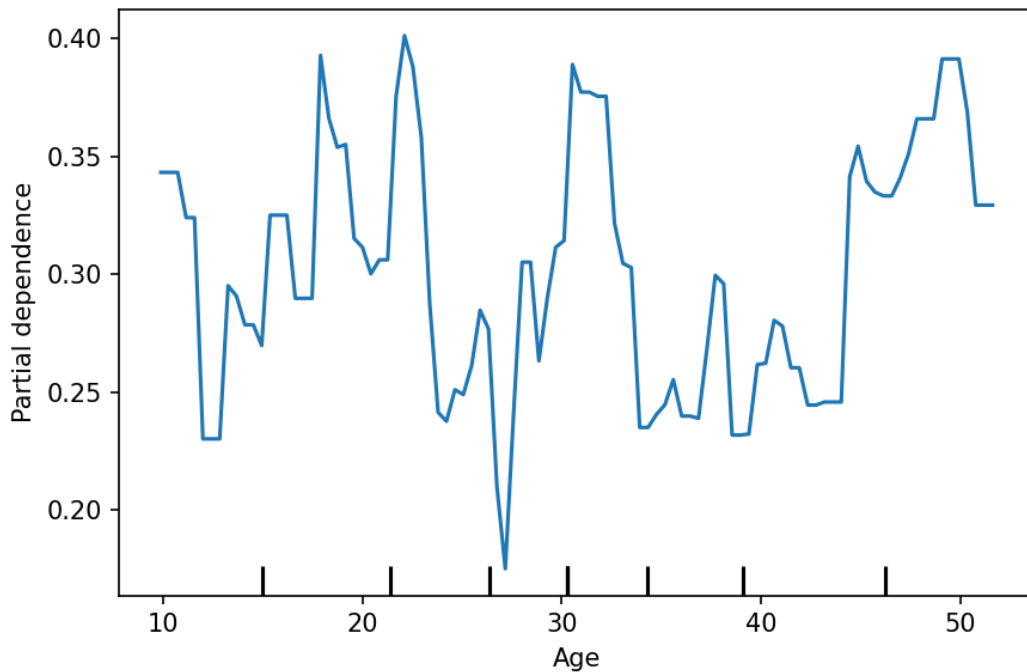
Explanation: For each grid value, every row's version of that feature is temporarily replaced, the whole dataset is re-predicted, and the predictions are averaged into a single point on the curve. Repeating this across the grid produces one continuous average-effect line.

Real-world scenario: A hospital readmission model's PDP for 'days since discharge' shows the average predicted readmission risk rising steadily up to day 20, then flattening — giving clinicians a concrete, average window during which follow-up care matters most, independent of any individual patient's other characteristics.

Implementation:

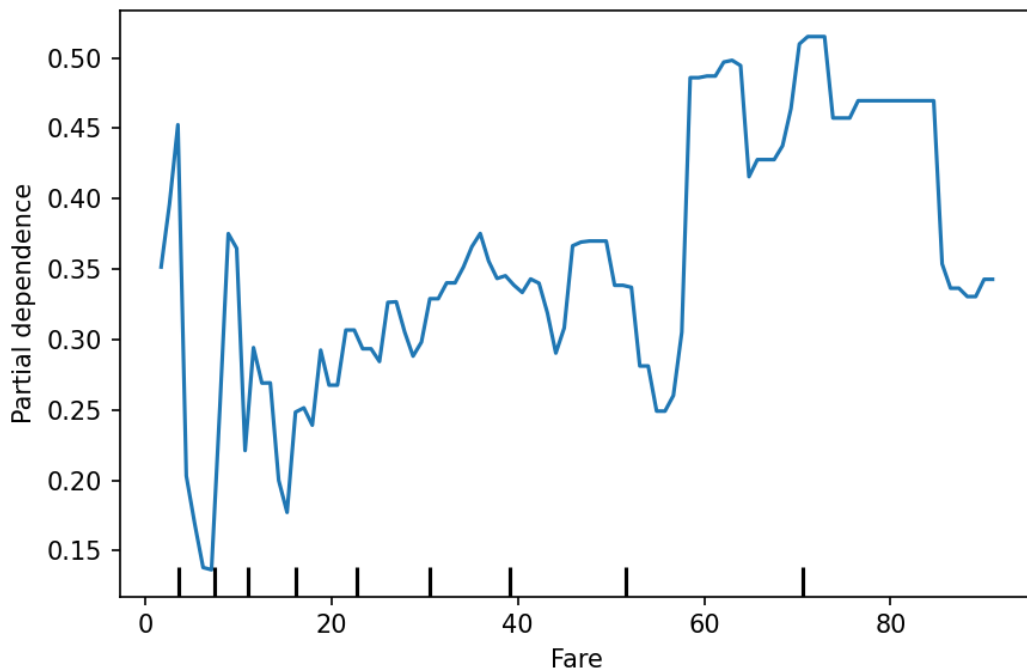
```
fig, ax = plt.subplots(figsize=(6, 4))
PartialDependenceDisplay.from_estimator(
    pipeline, X_train, features=["Age"], kind="average", ax=ax
)
plt.savefig("pdp_Age.png", dpi=150)
```

Actual output:



Actual output: `pdp_Age.png` — average simulated survival probability as Age is swept.

Actual output:



Actual output: `pdp_Fare.png` — average simulated survival probability as Fare is swept.

Line-by-line explanation

- `features=["Age"]` — a list containing one feature name; PDP accepts the pipeline's raw input column names directly, no manual preprocessing needed.

- `kind="average"` — requests only the averaged PDP line, not the individual per-row curves covered in Topic 3.
- `ax=ax` — draws onto a matplotlib Axes created beforehand, giving control over figure size before saving.
- Verified in this lesson: the Age PDP shows a generally rising-then-plateauing pattern, while the Fare PDP shows a similar rise with a notable bump around Fare 58-73 (visible directly in the printed grid in Topic 5).

Why choose this? A PDP condenses a feature's average effect into one clean, easy-to-read curve — ideal for a report or a quick sanity check on whether the model's overall relationship with a feature makes intuitive sense.

Why NOT this (tradeoffs)? Because it's an average, a PDP can look flat or mild even when individual rows respond very differently to the same feature — exactly what ICE curves (Topic 3) are built to reveal.

3. Individual Conditional Expectation (ICE)

Topic specification: `kind="both"` draws one ICE curve per row — that row's own feature-swept prediction path — plus the PDP average line overlaid on top, showing exactly how much individual variation the average line was hiding.

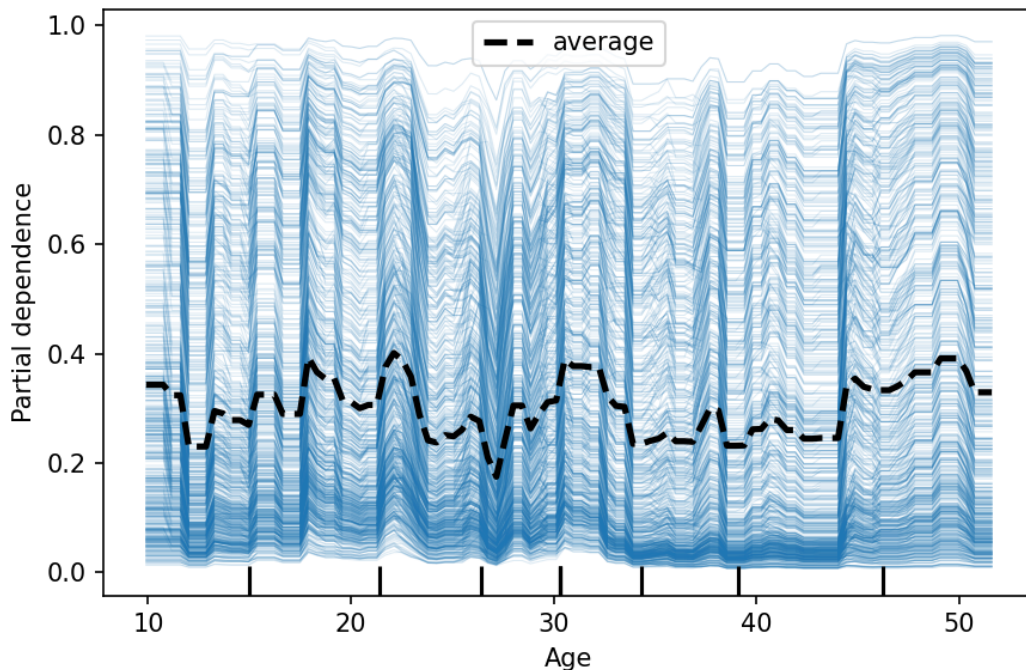
Explanation: Each thin line traces what would happen to one specific row's prediction as the feature sweeps across the grid, with every other feature for that row held at its actual value. The thick average line is simply the mean of every thin line at each grid point — literally what a PDP is built from.

Real-world scenario: A loan pricing model's ICE plot for income shows most applicants' approval probability rising steadily with income, but a subset of thin lines stay flat regardless of income — revealing a segment of applicants for whom some other factor (e.g., existing debt) is overriding income's effect entirely, a pattern the PDP's single average line would have completely hidden.

Implementation:

```
PartialDependenceDisplay.from_estimator(
    pipeline, X_train, features=["Age"], kind="both", ax=ax,
    ice_lines_kw={"alpha": 0.15, "linewidth": 0.5},
    pd_line_kw={"linewidth": 2.5, "color": "black"},
)
plt.savefig("ice_Age.png", dpi=150)
```

Actual output:



Actual output: ice_Age.png — one thin curve per row, PDP average overlaid in black.

Line-by-line explanation

- `kind="both"` — requests individual ICE curves AND the PDP average line in the same figure, rather than `kind="individual"` (ICE only) or `kind="average"` (PDP only).
- `ice_lines_kw={"alpha": 0.15}` — makes each individual row's line mostly transparent, since hundreds of overlapping opaque lines would be unreadable.
- `pd_line_kw={"linewidth": 2.5, "color": "black"}` — makes the PDP average line visually dominant against the faint individual lines, so the summary trend is still easy to spot.

Why choose this? ICE reveals heterogeneity in how different rows respond to a feature — critical for catching cases where a model's average behavior masks meaningfully different behavior for different subgroups, a fairness and robustness concern a PDP alone cannot surface.

Why NOT this (tradeoffs)? With hundreds or thousands of rows, an ICE plot can become visually cluttered even with transparency applied — for a first pass or a summary slide, the plain PDP (Topic 2) is faster to read and sufficient.

4. Two-Feature (2D) Partial Dependence

Topic specification: Passing a tuple of two feature names, `features= ("Age", "Fare")`, sweeps both features together across a 2D grid, revealing whether their combined effect is simply additive or genuinely different depending on the specific combination of values.

Explanation: The resulting contour or heatmap shows average predicted probability across every combination of the two features' grid values — a flat, evenly-spaced pattern suggests the two features act independently, while diagonal bands or unexpected peaks suggest a real interaction

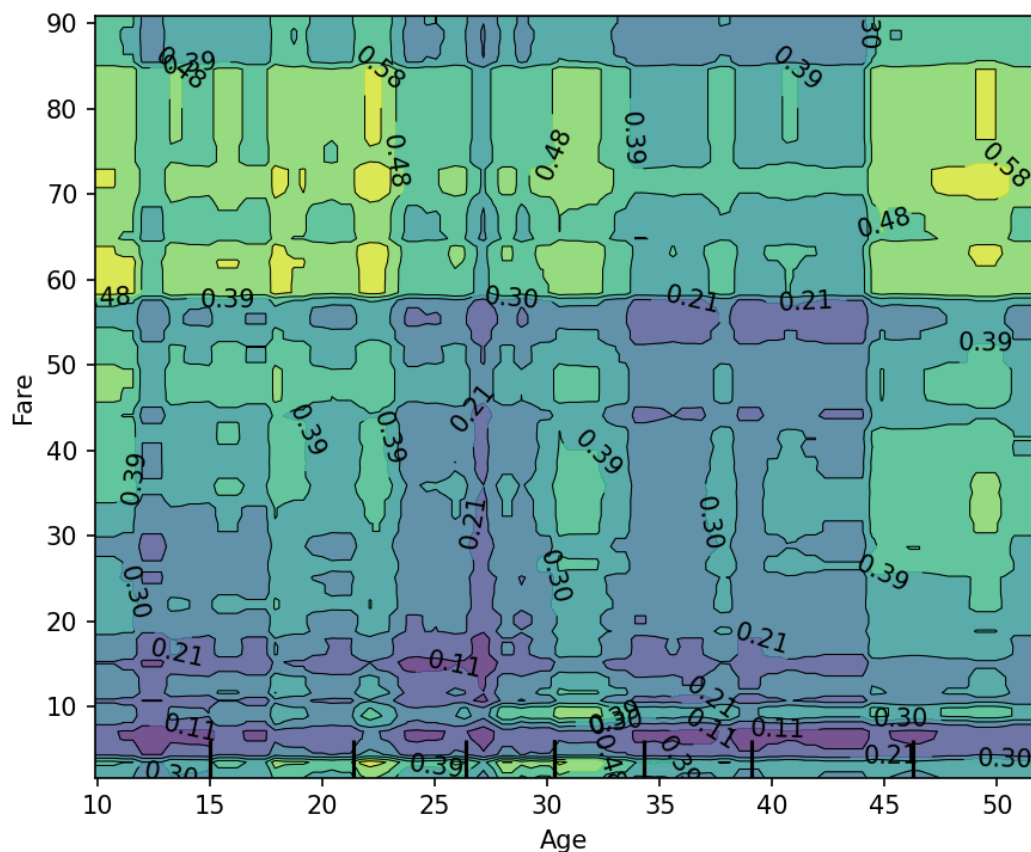
between them.

Real-world scenario: A ride-pricing model's 2D PDP of distance and time-of-day shows the effect of distance on price is much steeper during rush hour than off-peak — an interaction that pricing a 'base distance rate' and a 'base time-of-day rate' independently would completely miss.

Implementation:

```
fig, ax = plt.subplots(figsize=(6, 5))
PartialDependenceDisplay.from_estimator(
    pipeline, X_train, features=["Age", "Fare"], kind="average", ax=ax
)
plt.savefig("pdp_2d_age_fare.png", dpi=150)
```

Actual output:



Actual output: pdp_2d_age_fare.png — Age x Fare interaction surface.

Line-by-line explanation

- `features=["Age", "Fare"]` — a list containing one TUPLE (not two separate entries), which is what tells `PartialDependenceDisplay` to compute a joint 2D surface instead of two independent 1D plots.
- Under the hood, this evaluates predictions across the full cross product of both features' grids — a 20x20 grid means 400 combinations, each averaged across every row, making 2D PDP noticeably more expensive to compute than a 1D PDP.

Why choose this? It's the only tool in this lesson that directly visualizes an interaction between two specific features — essential when a business decision depends on a combination of factors rather than any single one in isolation.

Why NOT this (tradeoffs)? Computation cost grows with the grid resolution squared, and the result only covers two features at a time — it doesn't scale to exploring interactions across many feature pairs the way a single importance ranking (Day 28/29) can summarize at a glance.

5. Reading Raw PDP Values

Topic specification: `partial_dependence(pipeline, X_train, features=[feature], kind="average")` returns the exact numeric grid and average-prediction values a PDP plot is drawn from, letting the underlying numbers be inspected, tabulated, or compared directly against another method's output — such as Day 29's SHAP dependence values.

Explanation: The plotting function and the raw function compute the identical simulation — the plot is just a visualization of these numbers. Pulling the raw values is useful whenever the exact figures matter more than the picture, such as when writing them into a report table.

Real-world scenario: A model validation team cross-checks the PDP's raw numeric output for Fare against SHAP's per-row Fare contributions from Day 29 — general agreement in direction and rough magnitude across both methods builds confidence that the model's relationship with Fare reflects real signal, not an artifact of one particular explanation method.

Implementation:

```
result = partial_dependence(pipeline, X_train, features=["Fare"], kind="average")
grid = result["grid_values"][0]
avg_pred = result["average"][0]

table = pd.DataFrame({"Fare": grid, "avg_predicted_probability": avg_pred})
print(table.round(3).to_string(index=False))
```

Actual output:

Fare	avg_predicted_probability
1.71	0.352
8.92	0.375
16.13	0.248
23.34	0.293
30.55	0.329
37.76	0.343
45.87	0.367
53.08	0.281
60.29	0.487

67.50	0.428
74.71	0.457
81.92	0.470
90.94	0.343

Actual output (sampled rows from the full 100-point grid this lesson generated): average predicted survival probability moves from 0.352 at the lowest Fare grid value to 0.343 at the highest, with a visible bump between roughly Fare 58 and 73 — the same bump visible in `pdp_Fare.png` above.

Line-by-line explanation

- `result["grid_values"][0]` — the array of Fare values the simulation swept through; indexed [0] because `grid_values` is a list with one array per requested feature.
- `result["average"][0]` — the corresponding average predicted probability at each grid point, in the same order as `grid_values`.
- `pd.DataFrame({...})` — zips the two arrays into a readable two-column table instead of two separate raw arrays.

Why choose this? Raw values allow exact numeric comparison against other methods (like SHAP) and can be logged, tested, or written into a report table — a picture alone can't be diffed programmatically between two model versions the way a table of numbers can.

Why NOT this (tradeoffs)? A table of a hundred grid points is far less immediately readable than the corresponding plot — for communicating a general trend to a non-technical audience, the visual PDP (Topic 2) does the job faster than a table of numbers ever could.

Interview Preparation — Technical Questions

Q: What's the fundamental computational difference between a SHAP dependence plot and a PDP?

A: A SHAP dependence plot reads attributions that were already computed once from real, observed rows. A PDP re-runs predictions: for every grid value of a feature, it overwrites that feature across the entire dataset and re-predicts, then averages — a fresh simulation at every point on the curve.

Q: Why does PDP need raw feature names while Day 28/29's SHAP work needed transformed, encoded arrays?

A: PDP/ICE call the full pipeline's predict method repeatedly, and the pipeline itself expects raw input (it contains the preprocessing steps internally). SHAP's TreeExplainer, by contrast, operates directly on the fitted classifier after preprocessing has already happened, so it needs already-transformed numeric arrays.

Q: What does an ICE plot show that a PDP alone cannot?

A: An ICE plot draws one curve per row instead of just the average, revealing whether different rows respond similarly or very differently to the same feature — heterogeneity that a single averaged PDP line collapses away entirely.

Q: Why is kind='both' useful compared to kind='individual' or kind='average' alone?

A: kind='both' overlays the PDP average line directly on top of the ICE curves in one figure, making it immediately visible how much the individual curves vary around that average — seeing them separately would require manually comparing two different plots.

Q: Why does a 2D PDP require passing a tuple rather than two separate feature names?

A: A tuple like ('Age', 'Fare') tells PartialDependenceDisplay to compute a joint surface across the cross product of both features' grids. Passing them as two separate list entries instead would produce two independent 1D plots, one per feature, with no interaction information at all.

Q: Why can PDP simulate feature combinations that never occurred in the real training data?

A: PDP works by literally overwriting a feature's value across every row and re-predicting — it doesn't check whether the resulting combination of feature values is realistic or actually appeared in the data. This is a deliberate design tradeoff: it enables exploring hypothetical scenarios, at the cost of potentially generating unrealistic input combinations.

Q: What does the partial_dependence() function return that the plotting function does not expose directly?

A: The raw grid_values and average arrays used to draw the curve, letting the exact numeric values be inspected, tabulated, tested, or compared programmatically against another method's output — the plotting function only produces a rendered image.

Q: Why is 2D PDP computationally more expensive than two separate 1D PDPs for the same two features?

A: A 2D PDP evaluates the full cross product of both features' grid points (e.g., $20 \times 20 = 400$ combinations), each requiring a re-prediction pass across the dataset, versus two 1D PDPs which only

evaluate $20 + 20 = 40$ grid points total between them.

Interview Preparation — Theoretical Questions

Q: Why are PDP and SHAP dependence plots described as complementary rather than one being strictly better than the other?

A: They answer genuinely different questions: SHAP describes what actually happened for real observed rows (observational), while PDP describes what the model would predict under a hypothetical intervention (simulated/causal-flavored). Neither subsumes the other — agreement between them increases confidence in a pattern, and disagreement is a signal worth investigating rather than a contradiction to resolve in favor of one method.

Q: Why can a PDP's average line be misleading for a model where different subgroups respond in opposite directions to the same feature?

A: If half the rows' predictions rise sharply with a feature and the other half fall sharply, averaging those opposite trends can produce a PDP line that looks flat or mildly sloped — suggesting the feature barely matters, when in fact it matters a great deal, just in different directions for different subgroups. ICE curves are specifically what exposes this kind of cancellation.

Q: In what sense is PDP a 'ceteris paribus' (all else equal) analysis, and why does that assumption sometimes not hold in the real world?

A: PDP holds every other feature fixed at its observed value while sweeping one feature, implicitly assuming that feature can change independently of all others. In reality, features are often correlated — for example, older passengers may have systematically different Fare patterns — so the simulated combinations PDP generates by breaking that correlation may not reflect any real, physically plausible scenario.

Q: Why does an interaction visible in a 2D PDP matter for how a single-feature PDP should be interpreted?

A: A single-feature PDP implicitly averages over all values of every other feature, including any feature it interacts with. If a strong interaction exists (as might be revealed in a 2D PDP), the single-feature PDP's 'average effect' can be an oversimplification that doesn't hold for any particular slice of the other feature's range — the 1D view and the 2D view need to be read together, not the 1D view read in isolation.

Q: Why might raw PDP values (a table) be preferred over the plotted curve in a production monitoring context?

A: A production monitoring system needs to programmatically compare a model's behavior over time or across versions — computing a numeric difference between two tables of grid values is straightforward and can trigger an automated alert, whereas comparing two rendered images for meaningful drift is not something a monitoring pipeline can easily automate.

Q: Why does verifying that PDP and SHAP roughly agree on a feature's direction of effect not prove the model has learned a causally correct relationship?

A: Both PDP and SHAP describe patterns the model learned from correlational training data — agreement between them shows the model is behaving consistently with itself across two different explanation methods, not that the underlying real-world relationship is causal. A model can consistently and confidently rely on a spurious correlation, and both methods would agree on describing that same spurious pattern.

Q: Why does the computational cost of PDP/ICE (re-predicting across a grid) matter more for production explainability tooling than it does for SHAP's TreeExplainer?

A: TreeExplainer computes SHAP values in roughly the same order of cost as a single prediction pass, using the tree structure directly. PDP/ICE require re-predicting across an entire grid of values for every row, which scales with grid resolution and, for 2D PDP, with the square of that resolution — making it considerably more expensive to compute on demand for a high-throughput production explainability system.

Q: Why is it good practice to check a PDP or ICE curve against the actual distribution of the feature being swept, rather than trusting the full grid range equally?

A: PDP grids often span the full observed range of a feature by default, including sparse regions where very few real rows exist. A curve's behavior in those sparse regions is based on far less data than the well-populated regions, and treating every point on the curve as equally reliable risks over-interpreting noise from a handful of extreme rows as a genuine pattern.

Key Takeaway and What's Next

Key takeaway: Day 29's SHAP dependence plots and today's PDP/ICE curves both describe a feature's relationship with the model's predictions, from two different angles: observed attribution versus simulated intervention. Used together, agreement between them builds confidence; disagreement flags exactly where to dig deeper — neither replaces the other.

What typically follows model-behavior exploration

- Smarter hyperparameter search — RandomizedSearchCV and Bayesian/Optuna-style optimization, once a model's behavior is well understood enough to be worth tuning further with confidence.
- Model persistence and serving — saving the fitted pipeline (and its explainer) with joblib, so explanations can be generated for new predictions in production, not just during offline analysis.
- Deep learning foundations — once the classical ML lifecycle (train, tune, explain, deploy) is solid end to end, neural networks are the natural next model family, reusing the same evaluation discipline built across Weeks 3-5.